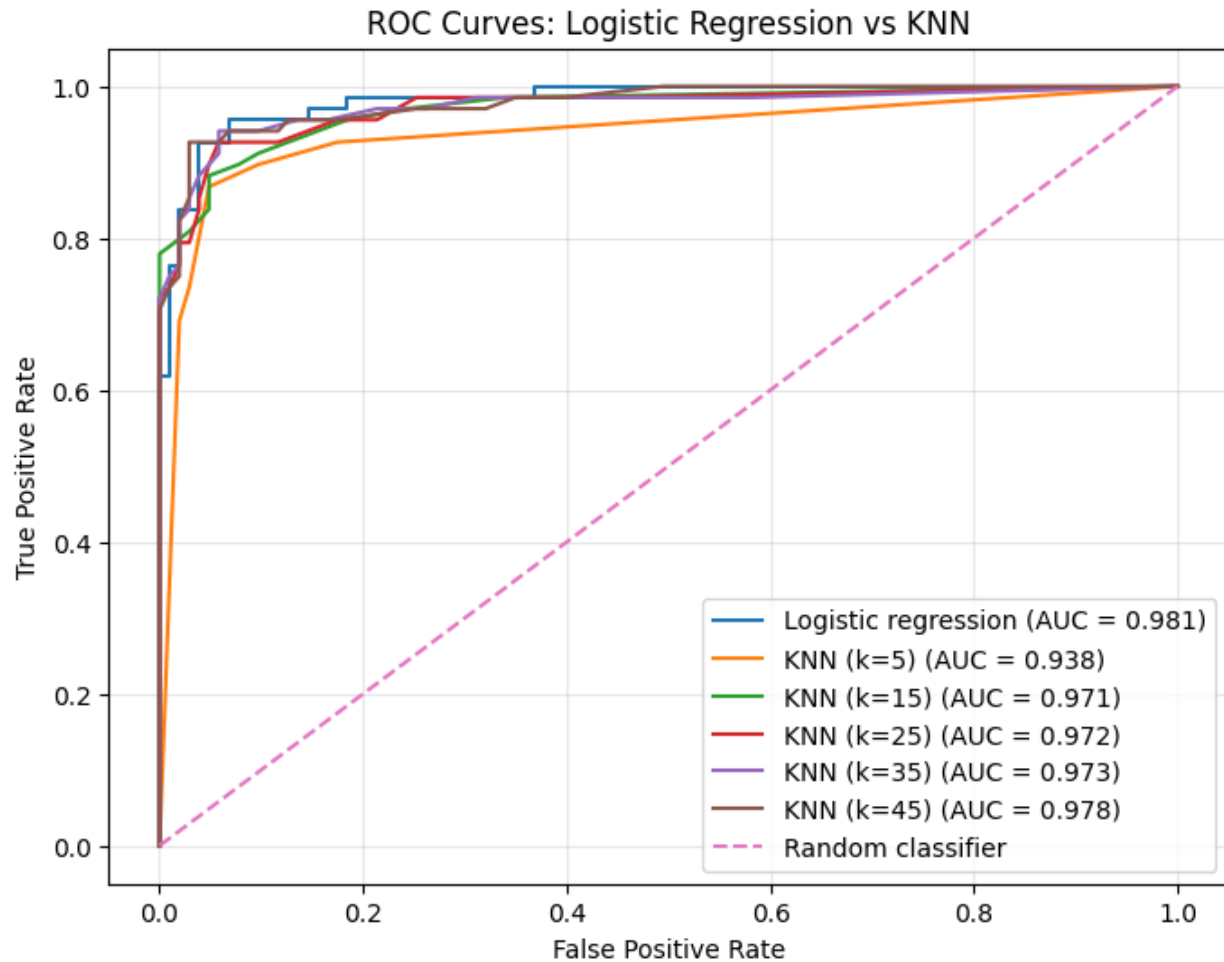


Question 1: Compare KNN and logistic regression



Logistic regression: AUC = 0.9814

KNN (k=5): AUC = 0.9377

KNN (k=15): AUC = 0.9706

KNN (k=25): AUC = 0.9717

KNN (k=35): AUC = 0.9732

KNN (k=45): AUC = 0.9780

I prefer logistic regression because it has the highest test-set AUC among the six models, indicating the best ability to discriminate between malignant and benign cases on the test data.

```

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import roc_curve, roc_auc_score

# =====
# 1. Read data
# =====

path = "/Users/runting/stat432-fall2026/homework/week-05/data/knn/breast-cancer.csv"
df = pd.read_csv(path)
print(df.head())
print(df.shape)

# =====
# 2. Select predictors and response
# =====

X = df[
    ["mean_radius", "mean_texture", "mean_smoothness"]
]
y = df["y"]

# =====
# 3. Split into 70% training and 30% test
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=123
)

# =====
# 4. Standardize
#
# IMPORTANT:
# The mean and SD are calculated ONLY from training data.
# The same transformation is then applied to test data.
# =====

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# =====
# 5. Logistic regression
# =====

logistic_model = LogisticRegression(
    penalty=None,
    fit_intercept=True,
    max_iter=1000
)

logistic_model.fit(X_train_scaled, y_train)

# Probability that y = 1
logistic_prob = logistic_model.predict_proba(
    X_test_scaled
)[: , 1]

# =====
# 6. KNN models
# =====

k_values = [5, 15, 25, 35, 45]
knn_prob = {}

for k in k_values:
    model = KNeighborsClassifier(
        n_neighbors=k,
        metric="euclidean",
        weights="uniform"
    )

```

```

    )

    model.fit(X_train_scaled, y_train)

    # Probability that y = 1
    knn_prob[k] = model.predict_proba(
        X_test_scaled
    )[:, 1]

# =====
# 7. ROC curves and AUC
# =====

fpr_logistic, tpr_logistic, _ = roc_curve(
    y_test,
    logistic_prob
)

auc_logistic = roc_auc_score(
    y_test,
    logistic_prob
)

roc_results = {
    "Logistic regression": {
        "fpr": fpr_logistic,
        "tpr": tpr_logistic,
        "auc": auc_logistic
    }
}

for k in k_values:
    fpr, tpr, _ = roc_curve(
        y_test,
        knn_prob[k]
    )

    auc = roc_auc_score(
        y_test,
        knn_prob[k]
    )

    roc_results[f"KNN (k={k})"] = {
        "fpr": fpr,
        "tpr": tpr,
        "auc": auc
    }

# =====
# 8. Print AUC values
# =====

for model_name, result in roc_results.items():
    print(
        f"{model_name}: AUC = {result['auc']:.4f}"
    )

# =====
# 9. Plot all six ROC curves
# =====

plt.figure(figsize=(8, 6))

for model_name, result in roc_results.items():
    plt.plot(
        result["fpr"],
        result["tpr"],
        label=f"{model_name} (AUC = {result['auc']:.3f})"
    )

# Random classifier
plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random classifier"
)

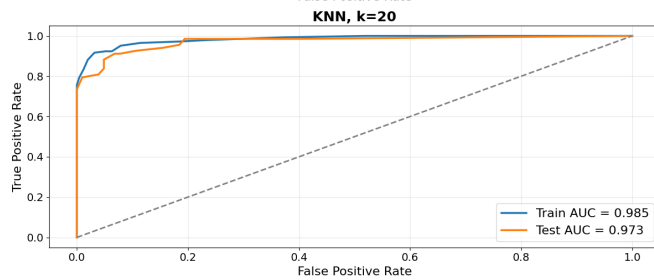
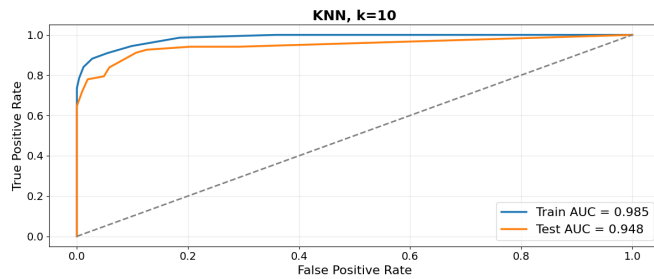
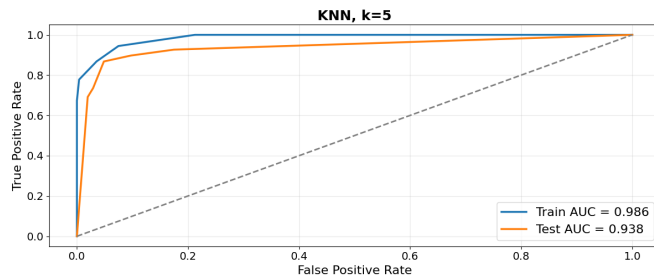
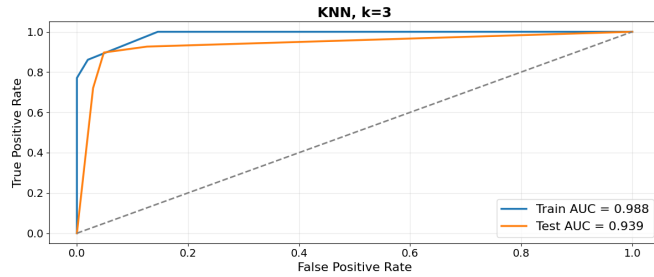
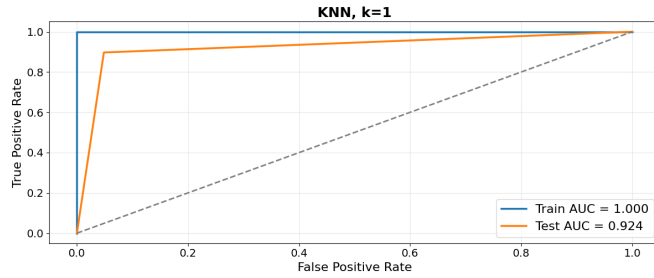
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves: Logistic Regression vs KNN")
plt.legend()

```

```
plt.grid(alpha=0.3)  
plt.show()
```

Question 2: Training and test ROC curves

k	Training AUC	Test AUC
1	1.000000	0.924258
3	0.987628	0.938678
5	0.985824	0.937750
10	0.985154	0.947744
20	0.984553	0.972587



Training performance measures how well the model fits observations it has already seen, while test performance measures how well the fitted model **generalizes to unseen observations**.

Training performance is generally higher than test performance because the KNN model is evaluated on observations that it has already seen during training. At $k=1$, each training observation is its own nearest neighbor, so the model perfectly reproduces the training labels and has an AUC of 1.

As k increases, the model becomes less flexible, so training performance generally decreases. Test performance may initially improve because the model becomes less prone to **overfitting**, although very large values of k can eventually lead to underfitting.

Thus, training performance may not reflect performance on new observations because high training performance can result from memorizing the training data rather than learning patterns that generalize.

Summary:

Small $k \rightarrow$ very high training performance, but potentially a larger training–test gap.

Increasing $k \rightarrow$ training performance tends to decrease because the model becomes less flexible.

Test performance may initially improve because the model becomes less sensitive to individual training observations.

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import roc_curve, roc_auc_score

# =====
# 1. Read data
# =====

path = "/Users/runting/stat432-fall2026/homework/week-06/data/breast-cancer.csv"
df = pd.read_csv(path)
X = df[
    ["mean_radius", "mean_texture", "mean_smoothness"]
]
y = df["y"]

# =====
# 2. Same 70/30 train-test split as Question 1
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=123
)

# =====
# 3. Same standardization as Question 1
#
# Fit scaler ONLY on training data.
# Apply the training transformation to both datasets.
```

```

# =====
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# =====
# 4. KNN with k = 1, 3, 5, 10, 20
# =====

k_values = [1, 3, 5, 10, 20]
results = {}

for k in k_values:
    model = KNeighborsClassifier(
        n_neighbors=k,
        metric="euclidean",
        weights="uniform"
    )

    model.fit(X_train_scaled, y_train)

    # -----
    # Training probabilities
    # The training observation itself IS included among
    # its k nearest neighbors.
    # -----

    train_prob = model.predict_proba(
        X_train_scaled
    )[:, 1]

    # -----
    # Test probabilities
    # -----

    test_prob = model.predict_proba(
        X_test_scaled
    )[:, 1]

    # -----
    # ROC
    # -----

    train_fpr, train_tpr, _ = roc_curve(
        y_train,
        train_prob
    )

    test_fpr, test_tpr, _ = roc_curve(
        y_test,
        test_prob
    )

    # -----
    # AUC
    # -----

    train_auc = roc_auc_score(
        y_train,
        train_prob
    )

    test_auc = roc_auc_score(
        y_test,
        test_prob
    )

    results[k] = {
        "train_fpr": train_fpr,
        "train_tpr": train_tpr,
        "train_auc": train_auc,
        "test_fpr": test_fpr,
        "test_tpr": test_tpr,
        "test_auc": test_auc
    }

# =====
# 5. Report AUC values
# =====

auc_table = pd.DataFrame({
    "k": k_values,

```

```

    "Training AUC": [
        results[k]["train_auc"] for k in k_values
    ]
    "test AUC": [
        results[k]["test_auc"] for k in k_values
    ]
})

print(auc_table)
# =====
# 6. Plot training and test ROC curves (Fixed font size)
# =====

fig, axes = plt.subplots(
    5, 1,
    figsize=(12, 25) # Adjusted width-to-height ratio for 5 stacked subplots
)

for ax, k in zip(axes, k_values):
    result = results[k]

    # Training ROC
    ax.plot(
        result["train_fpr"],
        result["train_tpr"],
        label=f"Train AUC = {result['train_auc']:.3f}",
        linewidth=2.5
    )

    # Test ROC
    ax.plot(
        result["test_fpr"],
        result["test_tpr"],
        label=f"Test AUC = {result['test_auc']:.3f}",
        linewidth=2.5
    )

    # Random classifier
    ax.plot(
        [0, 1],
        [0, 1],
        linestyle="--",
        color="gray",
        linewidth=2
    )

    ax.set_title(f"KNN, k={k}", fontsize=18, fontweight="bold")
    ax.set_xlabel("False Positive Rate", fontsize=16)
    ax.set_ylabel("True Positive Rate", fontsize=16)
    ax.tick_params(axis='both', labelsize=14)
    ax.legend(fontsize=16, loc="lower right")
    ax.grid(alpha=0.3)

plt.tight_layout()
plt.show()

```

Question 3: Peer evaluation of Homework 5

Peer evaluation 1: Student rahafaa2, Question 5

<https://github.com/rahafala/stat432-fall2026/issues/2>

Advice — Q5, missing reflection

1. The personal reflection on the revised plot is missing entirely.
2. The prompt clearly asks "do you like the changes? Note anything you'd revise," but the answer just leaves the italic prompt text there with nothing written. It looks like this part was skipped.
3. *Correction:* Add a short reflection, e.g.: "The revised plot is cleaner overall — the bold title and removed top/right borders reduce visual clutter. I would keep the colorblind-

friendly palette. One thing I might adjust is the y-axis label 'Error (squared units)' — since the three curves are bias², variance, and MSE, simply 'Value' is actually fine, but adding units like '(squared response units)' makes it more precise."

Peer evaluation 2: Student cgao19, Question 5

<https://github.com/Gicooaidun/stat432-fall2026/issues/3>

Advice — Q1(b), degrees-of-freedom comparison

1. The n/k vs. ridge hat matrix comparison is dropped without explanation.
2. It's a smart point but reads like a name-drop without telling the reader what it means.
3. *Correction:* Add one clarifying sentence: "Both KNN and ridge have a notion of effective model complexity — for KNN it's roughly n/k (larger k means fewer effective parameters), and for ridge it's the trace of the hat matrix (larger λ means fewer effective parameters). This makes the bias-variance trade-off in both methods structurally the same, just controlled by different knobs."